

Fine-tuning LLMs with TRL and OpenEnv

Sergio Paniego Blanco ([@sergiopaniego](https://twitter.com/sergiopaniego))
Machine Learning Engineer @ Hugging Face
Slides: <https://github.com/sergiopaniego/talks>



Quick intro to Hugging Face 🤗

AI open platform for building, sharing and deploying AI (HF Hub):

- Models, datasets, apps...
- Posts, blogs, courses, papers, leaderboards...

The screenshot displays the Hugging Face website interface. At the top, there's a navigation bar with links for Models, Datasets, Spaces, Community, Docs, Enterprise, and Pricing. Below this, the main content area is divided into three columns. The left column contains a sidebar with a 'New' button, a user profile for 'sergiopaniago', and sections for 'Organizations' and 'Resources'. The middle column shows a 'Following' feed with updates from users like 'Steveeeeeen' and 'Ethank01'. The right column features a 'Trending' section with popular models such as 'A11baba-NLP/Tongyi-DeepResearch-30B-A3B' and 'ibm-granite/granite-docling-258M'. The bottom of the page includes a footer with 'Light theme' and a 'New Post' button.

2.6M

public models

800K+

public datasets

1M+

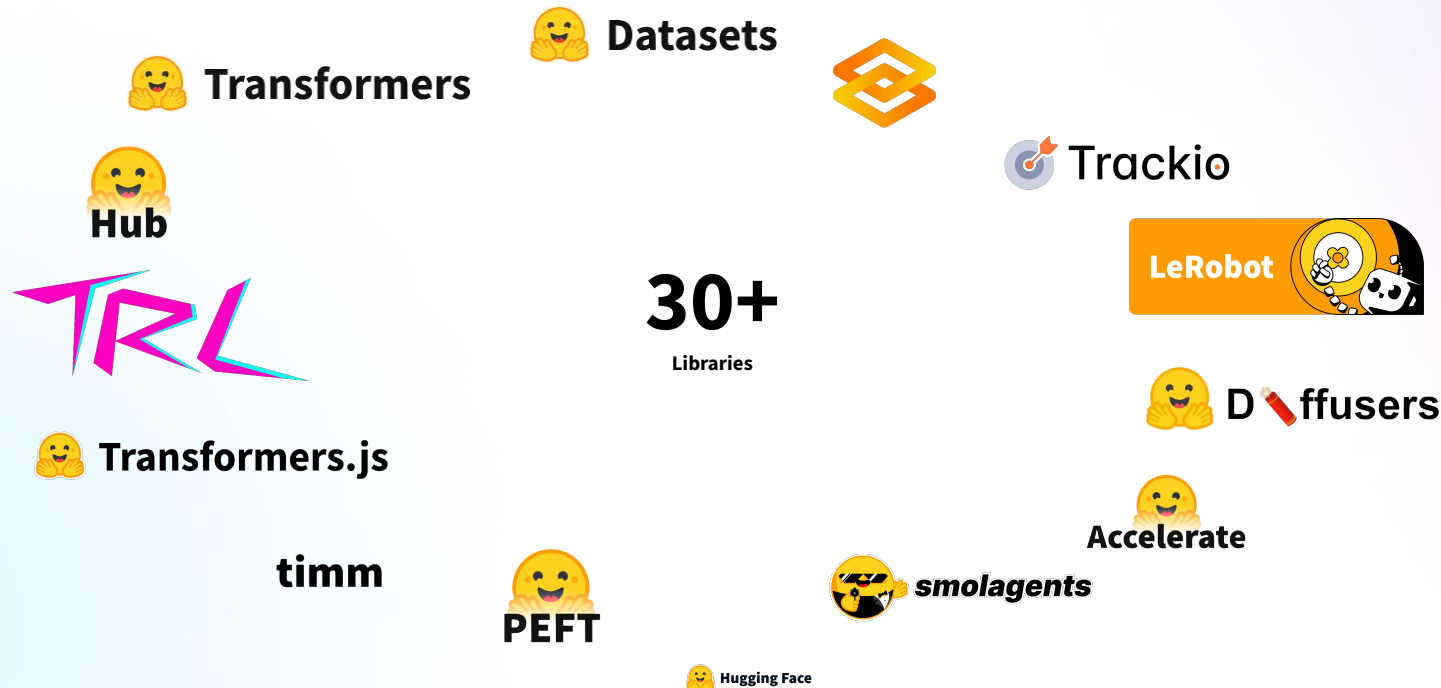
daily downloads

1M+

daily visitors

Quick intro to Hugging Face

Set of open source projects to empower AI development and advancement



Quick intro to Hugging Face

Find model → **Customize** → Deploy



Transformers



From fine-tuning to TRL

(fine tuning+alignment)

Classic fine-tuning teaches the model to **reproduce patterns in your data**, but not necessarily to **behave in ways you want** (human preferences, reasoning, agentic capabilities...)

- Dataset ($X \rightarrow Y$) $\rightarrow$ Pretrained model $\rightarrow$ Fine-tuned model
- Optimizes likelihood: predicts correctly based on data

We may need something stronger $\rightarrow$ **TRL**

Where does TRL fit in the model lifecycle?

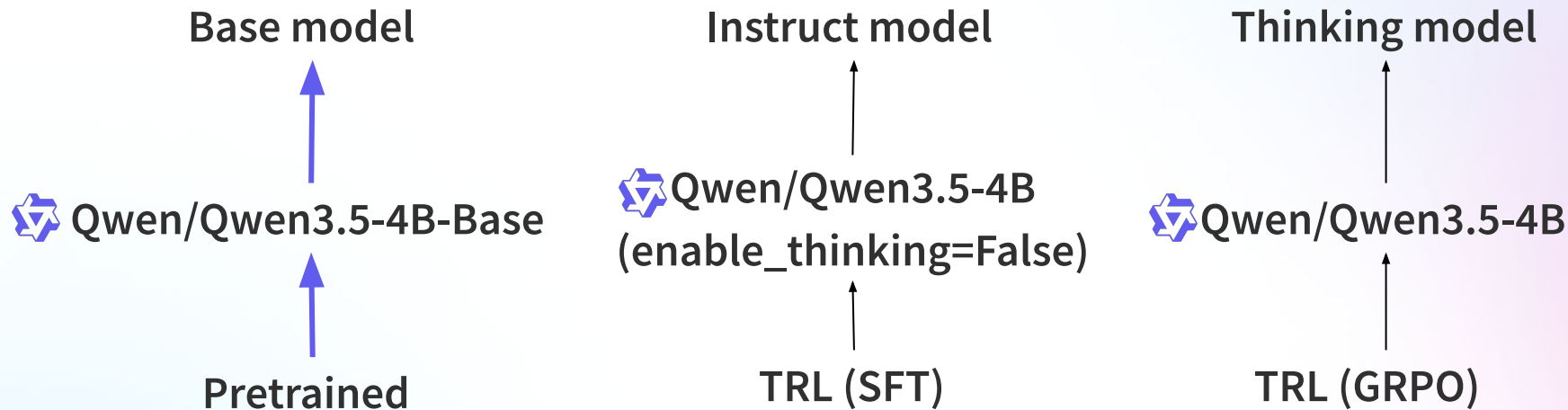
Pre-training: teaches general capacities such as language use, broad reasoning, and world knowledge

Mid-training: imparts domain knowledge (code, medial, databases, internal docs...)

Post-training/Alignment (*TRL*): learns to behave as we want (instruction following, reasoning, agentic behaviors...)

TRL operates in post-training → where we optimize behavior, not just likelihood

From token prediction to agentic intelligence



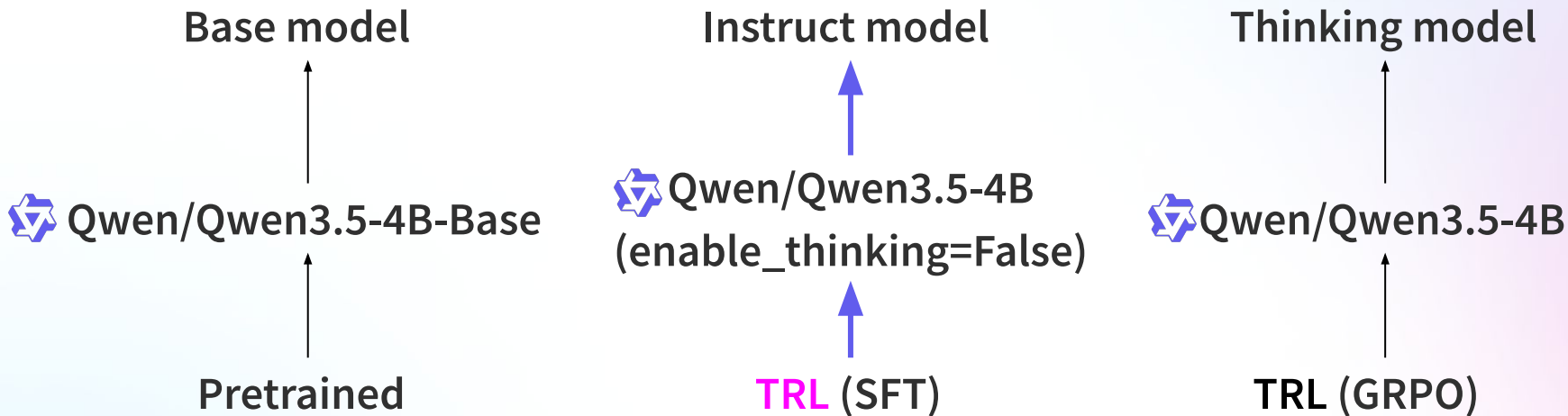
Base: predicts next token

Raw text, high quantity of data, masked language modeling, industrial compute

“Translate this sentence → Translate this sentence into...”

Optimizes: maximize $P(\text{next_token} \mid \text{context})$

From token prediction to agentic intelligence



Instruct: follows instructions

Structures responses, high quality data, reduced compute

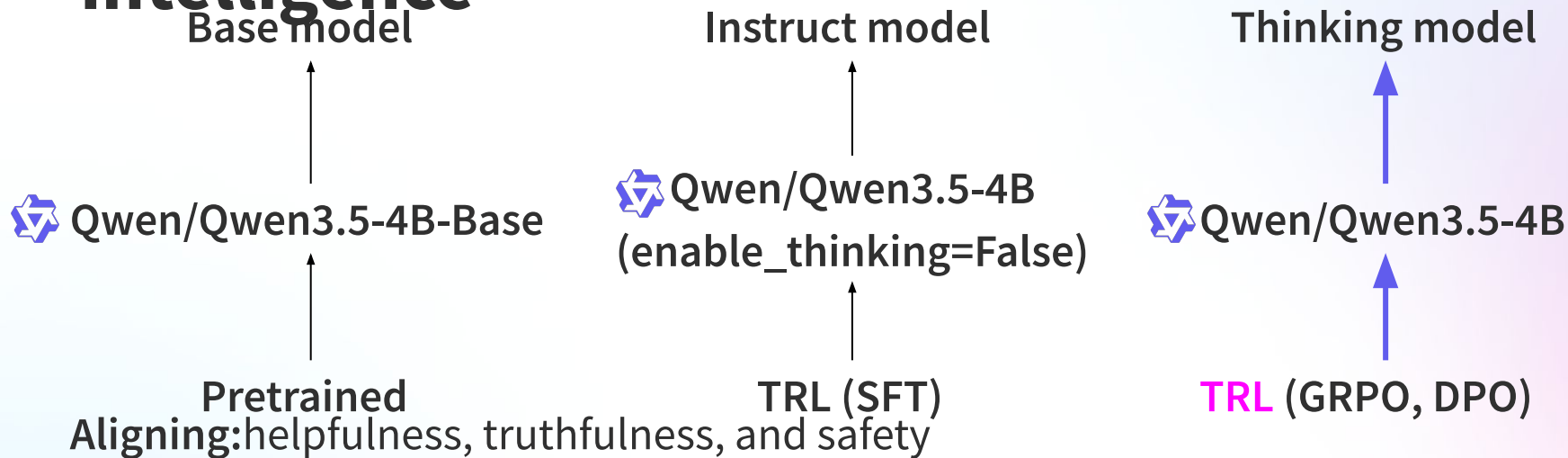
"<|user|>Translate this sentence</s>

<|assistant|>Sure! Here's the translation</s>

[Example dataset](#)

Optimizes: maximize $P(\text{response} \mid \text{instruction})$

From token prediction to agentic intelligence



Preferences, high quality, preference optim, consumer compute

“<|user|>How many helicopters can a human eat?</s>



<|assistant|>Human cannot eat helicopters</s>

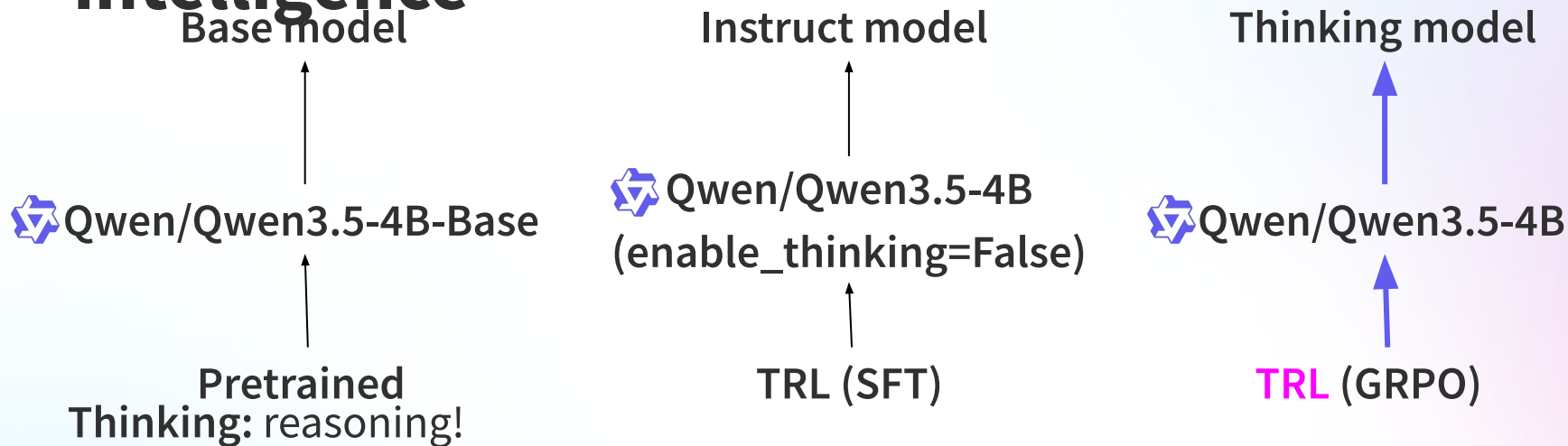


<|assistant|>That's not possible</s>

[Example dataset](#)

Optimizes: prefer chosen response over rejected response

From token prediction to agentic intelligence



"<|user|>How many helicopters can a human eat?</s>

<|assistant|>

<think>Tricky question. Let's break it down...</think>

That's not possible! I found that...</s>

[Example dataset](#)

Optimizes: maximize reward for reasoning trajectory

From token prediction to agentic intelligence

From reasoning to Acting: reasoning is internal optimization.

With GRPO, we reward *how* the model thinks → Can we reward what it *does*?

From:

Prompt → <think> ... </think> → Answer → Reward

To:

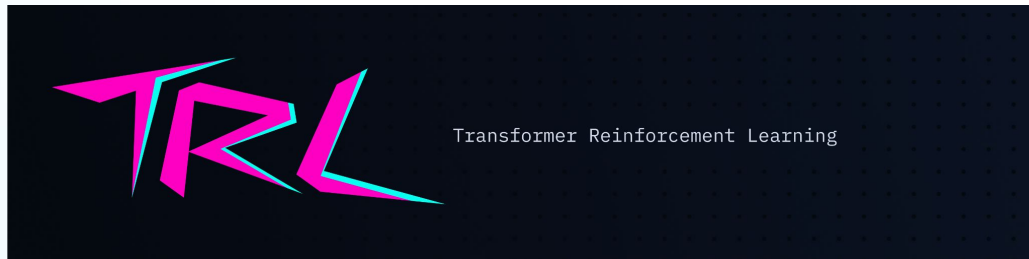
State → Action → Environment → Reward → New State → ...

More on this later → OpenEnv!

TRL: Fine-tuning, alignment & reasoning

Library that offers specialized trainers (SFT, GRPO, PPO, DPO...)

- Bridges the gap between **instruction following** and **full alignment pipelines**
- Supports: **accelerate**, **PEFT**, **kernels**, **vLLM**...
- **Multimodal** models support
- **Focus**: post-training → SFT, alignment (DPO), reasoning (GRPO)



Trainer vs TRL

TRL trainers are **built on top of transformers Trainer**, enhancing its functionality

Each trainer **abstracts a full training method**, so you don't need to reimplement everything from scratch

Gives ML engineers **ready-to-use recipes** for SFT, GRPO, PPO, DPO...

Focus: practicality, reproducibility, and reduced boilerplate

```
from trl import SFTTrainer
from datasets import load_dataset

trainer = SFTTrainer(
    model="Qwen/Qwen3-0.6B",
    train_dataset=load_dataset("trl-lib/Capybara", split="train"),
)
trainer.train()
```

TRL taxonomy

Online methods

(learn from feedback) Model generates responses and learns from reward signals during training

- GRPO ⚡
- RLOO ⚡
- OnlineDPO ⚡
- NashMD ⚡
- XPO ⚡
- PPO

Reward modeling

Training a model to judge responses

- PRM
- Reward

⚡ ⇒ LLM support

Offline methods

(data-driven) Model learns from human preference datasets or supervised signals

- SFT
- DPO
- ORPO
- BCO
- CPO
- KTO

Knowledge distillation

Transferring knowledge from a stronger model to a smaller one

- GKD

SFT and GRPO

We'll focus here on two trainers, but there are more

Supervised Fine-Tuning (SFT)

- Teach the model to follow instructions
- Uses human-labeled data (prompt → response)
- Improves helpfulness

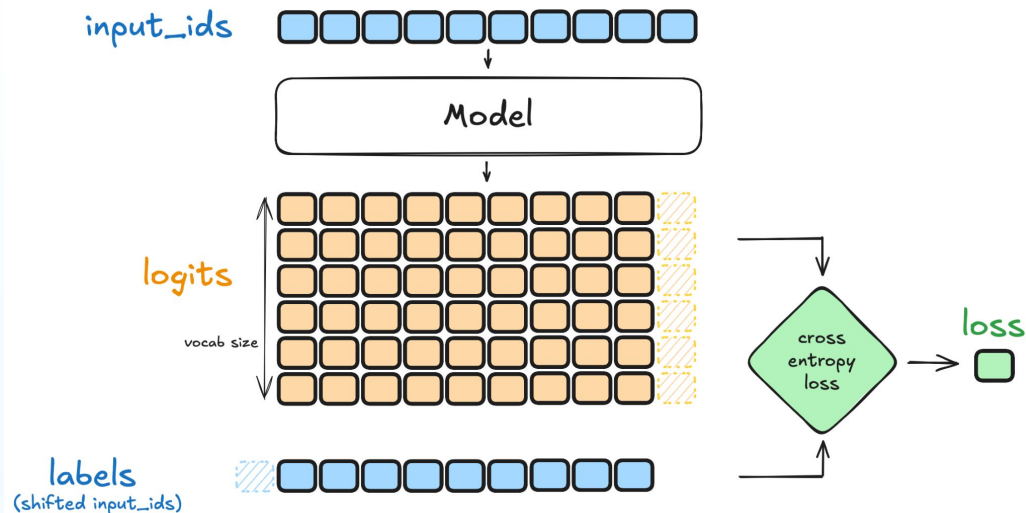
Group Relative Policy Optimization

- Train with online RL
- Uses reward functions
- Improves alignment and reasoning

Supervised Fine-Tuning

SFT is the simplest and most commonly used method for adapting an LLM to a target dataset

- Goal: minimize negative log-likelihood of a sequence
- Simple, stable, and super effective



SFT is easy to instantiate and train

Only needs a model (even just the name!) and a dataset from the Hub to train

```
from trl import SFTTrainer
from datasets import load_dataset

trainer = SFTTrainer(
    model="Qwen/Qwen3-0.6B",
    train_dataset=load_dataset("trl-lib/Capybara", split="train"),
)
trainer.train()
```

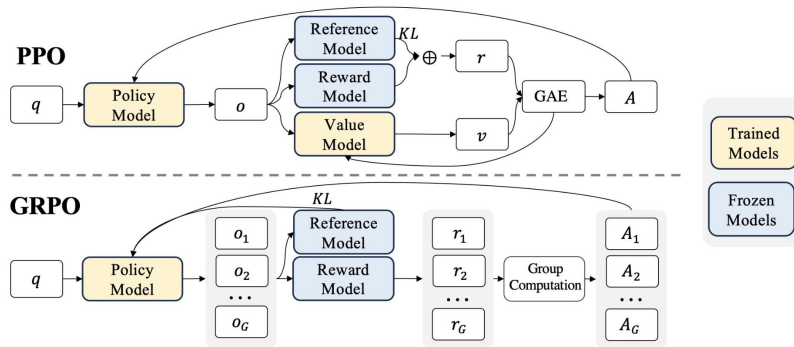
Group Relative Policy Optimization

GRPO was described in [DeepSeekMath](#) paper

Variant of Proximal Policy Optimization (PPO), that enhances maths reasoning capabilities while optimizing memory usage.

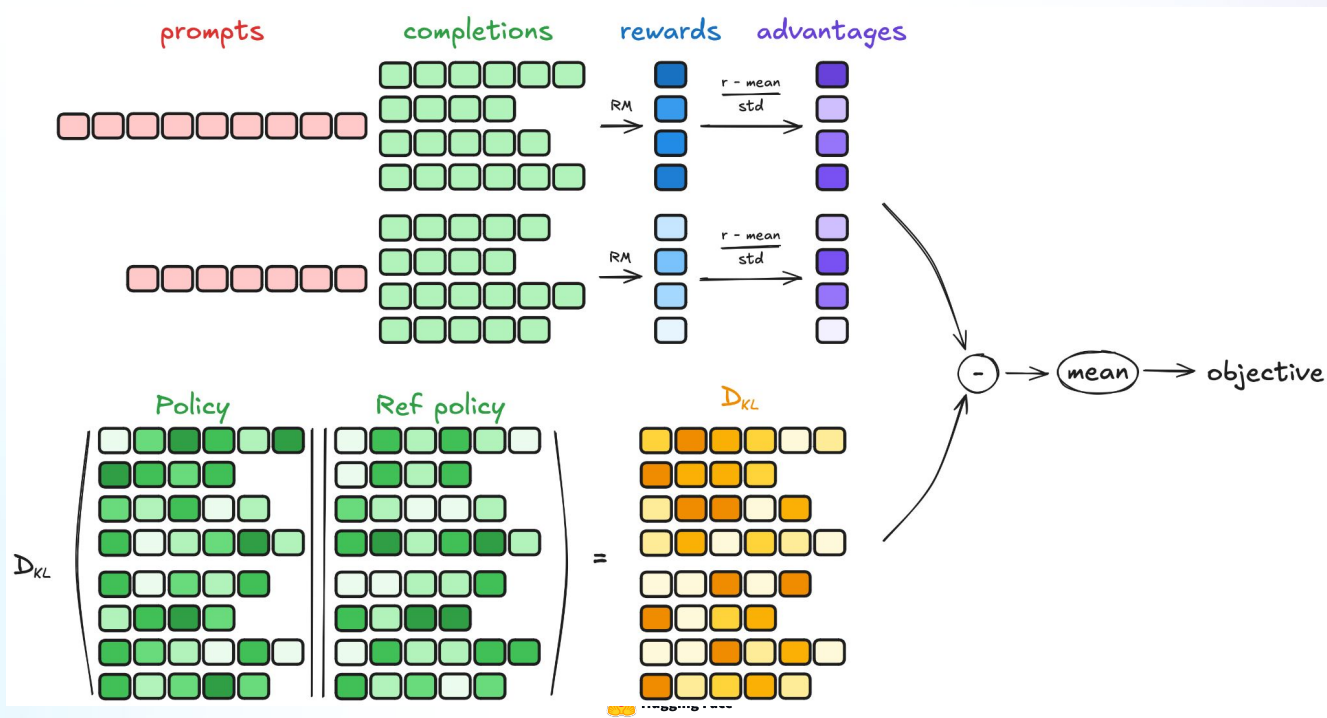
- Online method
- Optimized behavior using reward functions

Successor of PPO: more stable and memory efficient



GRPO

Steps: generating completions, computing advantage, computing the loss



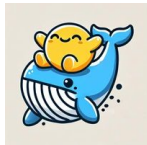
GRPOTrainer and GRPOConfig

```
from trl import GRPOTrainer, GRPOConfig
import re

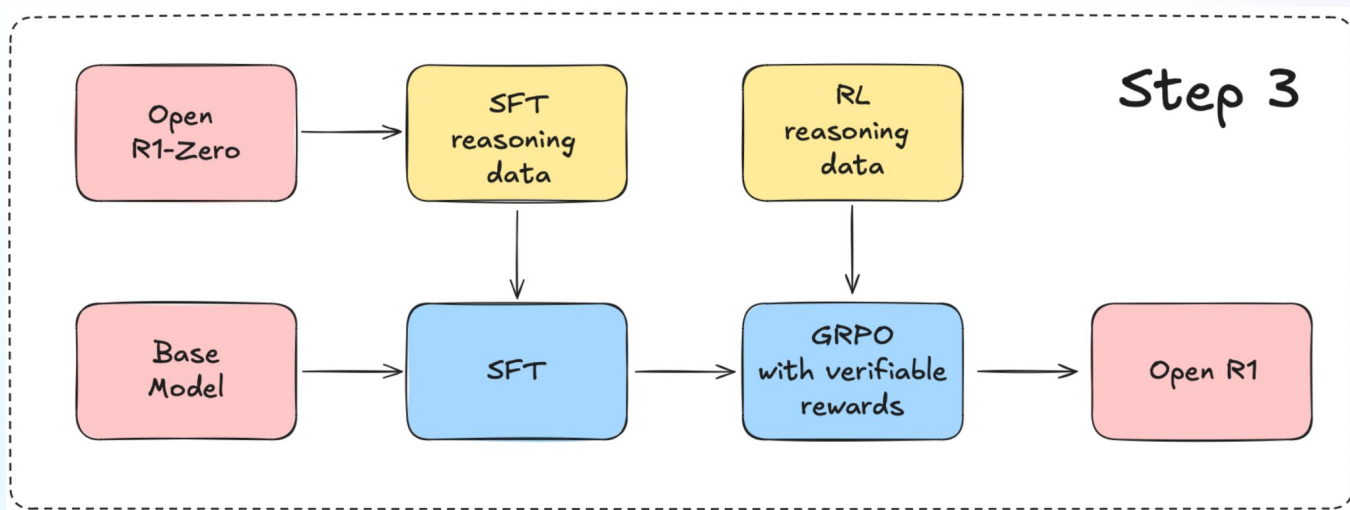
def format_reward_func(completions, **kwargs):
    """Reward function that checks if the completion has a specific format."""
    pattern = r"^<think>.*?</think><answer>.*?</answer>$"
    completion_contents = [completion[0]["content"] for completion in completions]
    matches = [re.match(pattern, content) for content in completion_contents]
    return [1.0 if match else 0.0 for match in matches]

trainer = GRPOTrainer(
    model="Qwen/Qwen2-0.5B-Instruct",
    reward_funcs=[format_reward_func, ... ],
    args=GRPOConfig( ... ),
    train_dataset=load_dataset("trl-lib/tldr", split="train"),
    # ...
)
trainer.train()
```

OpenR1

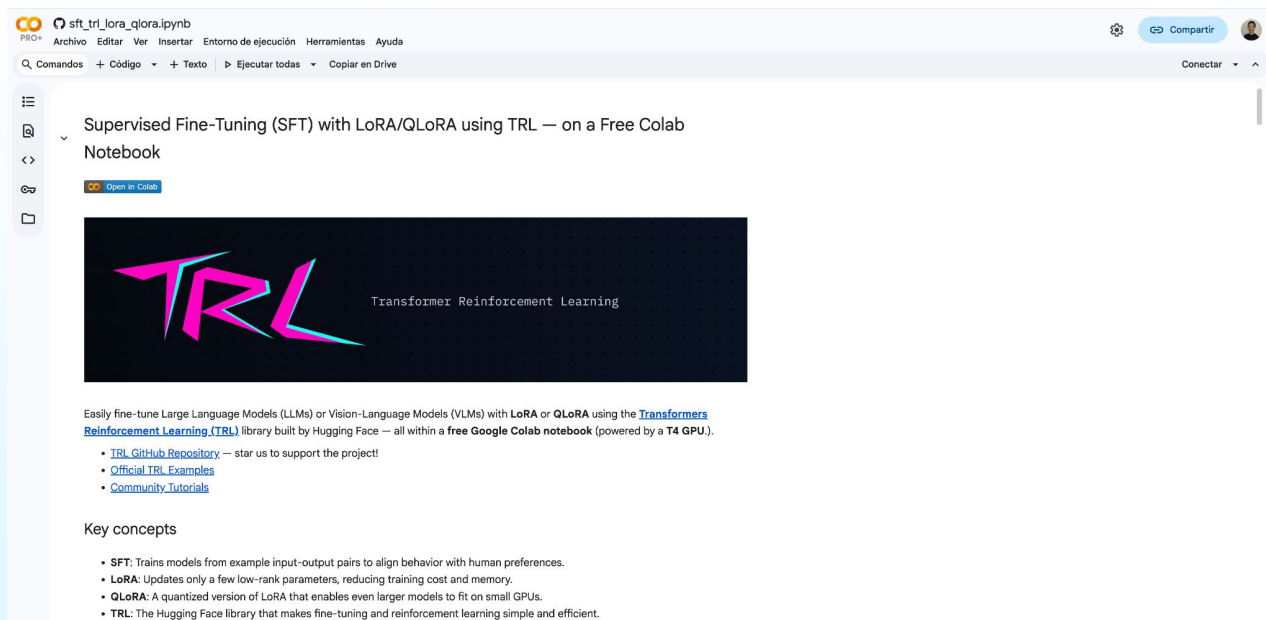


Initiative by HF to replicate and extend the techniques behind DeepSeek-R1 🐳
This model was built using SFT+GRPO!



Optimizing for GPU poors

We developed [notebooks](#) for training (14B) models in free Colab!



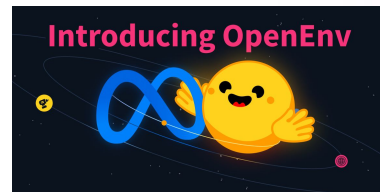
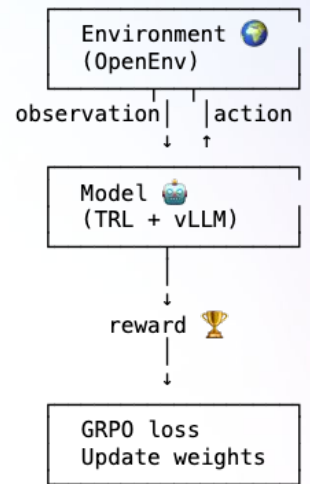
The screenshot shows a Google Colab notebook interface. The title bar indicates the notebook is named 'sft_trl_lora_qlora.ipynb'. The main content area features a large graphic with the text 'TRL Transformer Reinforcement Learning'. Below the graphic, there is a paragraph explaining that the notebook is for easily fine-tuning Large Language Models (LLMs) or Vision-Language Models (VLMs) with LoRA or QLoRA using the Transformers Reinforcement Learning (TRL) library. It also includes links to the TRL GitHub Repository, Official TRL Examples, and Community Tutorials. At the bottom, there is a section titled 'Key concepts' with a bulleted list:

- SFT: Trains models from example input-output pairs to align behavior with human preferences.
- LoRA: Updates only a few low-rank parameters, reducing training cost and memory.
- QLoRA: A quantized version of LoRA that enables even larger models to fit on small GPUs.
- TRL: The Hugging Face library that makes fine-tuning and reinforcement learning simple and efficient.



OpenEnv for training agents

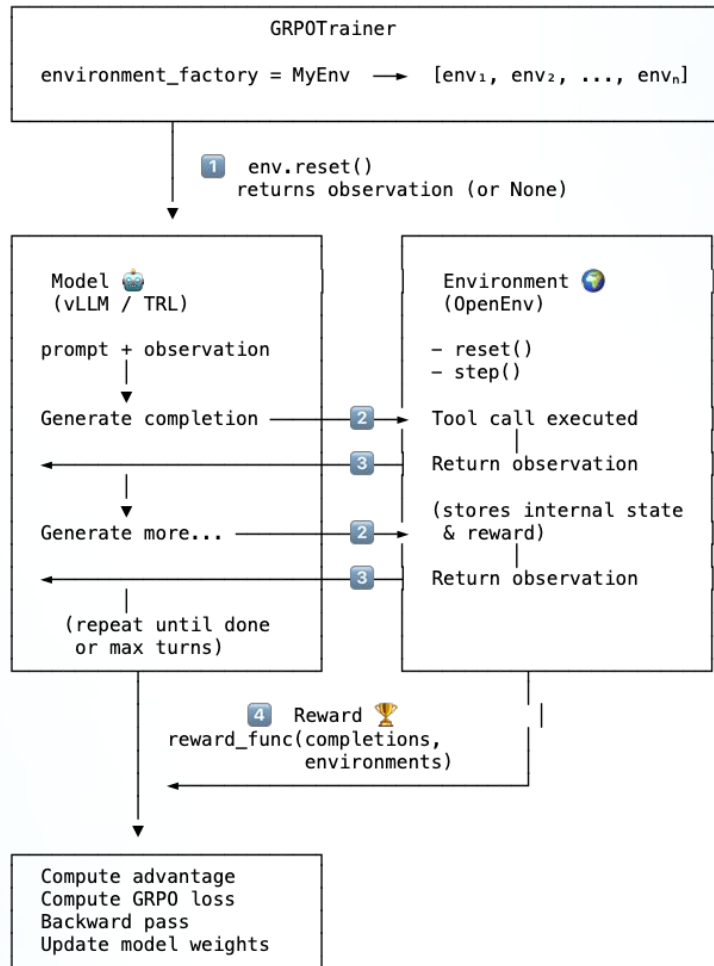
- So far: models learn from static datasets or simple reward functions
- Next step: **Models that learn by interacting with environments**
- Problem: every environment has its own API, format, logic → no standard
- [OpenEnv](#) → **unifies and integrates** with TRL (although TRL supports *any* framework)



OpenEnv for training agents

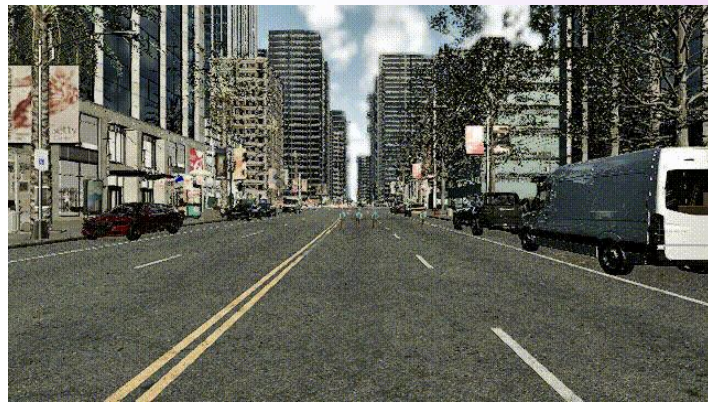
Framework for creating environments that can be used for training

- **Standardized:** Simple interface → `reset()` + public methods as tools
- **Shearable:** Public & discover envs of the Hub
- **Flexible:** Run locally, in Docker or on HF Spaces
- **Integrated:** Native integration with TRL's *GRPOTrainer*
- Some examples: browser, CARLA simulator, coding sandbox, games...



Example: learning to drive with CARLA

- [CARLA](#): autonomous driving simulator
- Scenario: vehicle heading towards pedestrians
- Model must observe, reason, and act to minimize harm
- Tools:
 - *observe()*: look at the scene
 - *emergency_stop()*: maximum braking
 - *lane_change(direction)*: swerve left/right
- Synchronous simulation
- Runs on HF Spaces, scalable with multiple instances



```

class CarlaEnv:

    def __init__(self):
        self.client = CarlaEnv(base_url="https://sergiopaniego-carla-env.hf.space")

    def reset(self, **kwargs) → str:
        result = self.client.reset(scenario_name="trolley_micro_escape_exists")
        self.reward = 0.0
        return describe(result.observation)

    def observe(self) → str:
        """Get the current scene description."""
        result = self._advance()
        self.reward = result.observation.rubric_reward or 0.0
        return describe(result.observation)

    def emergency_stop(self) → str:
        """Apply maximum braking to stop the vehicle."""
        self.client.step(CarlaAction(action_type="emergency_stop"))
        result = self._advance()
        self.reward = result.observation.rubric_reward or 0.0
        return describe(result.observation)

    def lane_change(self, direction: str) → str:
        """Change lane to avoid obstacles. Direction: 'left' or 'right'."""
        self.client.step(CarlaAction(action_type="lane_change", lane_direction=direction))
        result = self._advance()
        self.reward = result.observation.rubric_reward or 0.0
        return describe(result.observation)

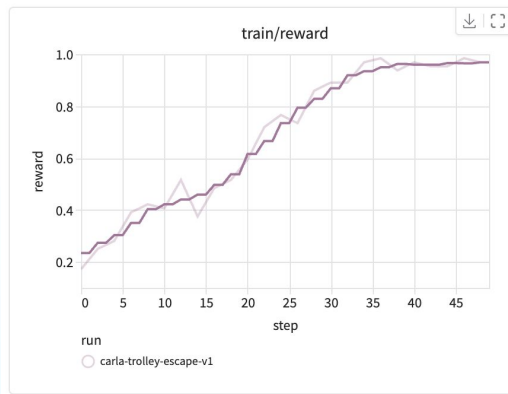
    def reward_func(completions, environments, **kwargs):
        return [env.reward for env in environments]

trainer = GRPOTrainer(
    model="Qwen/Qwen3-0.6B",
    reward_funcs=reward_func,
    environment_factory=CarlaEnv, # 3 tools auto-registered!
)
trainer.train()

```



CARLA: from crashing to avoiding

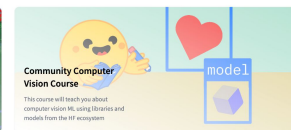
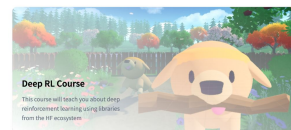
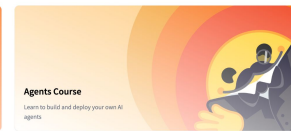
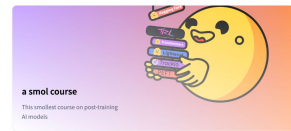
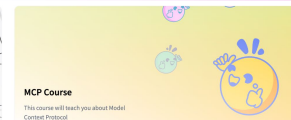
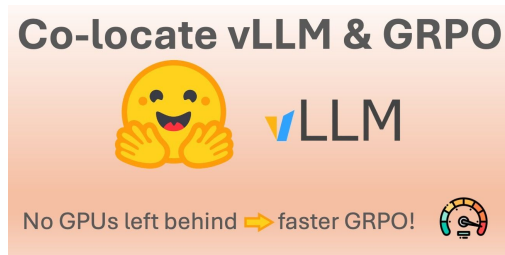
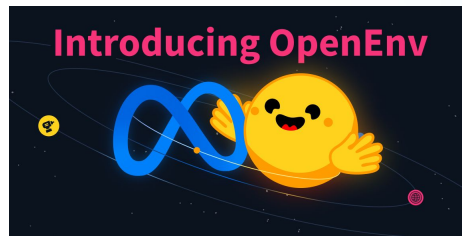


Qwen3-0.6B with GRPO

- Step 1 –  `observe()`
 - Speed: 39.6 km/h | 3 pedestrians at ~19m ahead
- Step 2 –  `emergency_stop()` # start braking to reduce speed
 - Speed: 27.4 km/h | pedestrians at ~14m
- Step 3 –  `lane_change("left")` # swerve while still slowing down
 - Speed: 27.6 km/h | pedestrians at ~10m (moving to adjacent lane)
- Step 4 –  `emergency_stop()` # full stop to ensure no collision
 - Speed: 0.0 km/h | pedestrians at ~6m (car stopped, collision avoided ✓)

Resources

- [Scripts](#)
- [Notebooks](#)
- [Cookbooks](#)
- [Example guides](#)
- [Blogs](#)
- [Courses](#)





Thank you

Sergio Paniego Blanco ([@sergiopaniego](https://twitter.com/sergiopaniego))

Machine Learning Engineer @ Hugging Face

Slides: <https://github.com/sergiopaniego/talks>

